

1. Flujos de caracteres (Reader y Writer)

A diferencia de los streams de bytes:

Aquí trabajamos con **caracteres (Unicode)**

Son ideales para texto (archivos `.txt`, `.csv`, etc.)

Clases principales:

- `Reader` → leer caracteres
 - `Writer` → escribir caracteres
-

2. Leer texto con FileReader

Ejemplo básico

```
import java.io.FileReader;  
import java.io.IOException;
```

```
public class LeerTexto {  
    public static void main(String[] args) {  
        try (FileReader fr = new FileReader("archivo.txt")) {  
  
            int caracter;  
  
            while ((caracter = fr.read()) != -1) {  
                System.out.print((char) caracter);  
            }  
  
        } catch (IOException e) {  
            System.out.println("Error al leer");  
        }  
    }  
}
```

Explicación:

- `read()` → lee un carácter
 - Devuelve `-1` al final
 - Convertimos a `char`
-

3. Escribir texto con FileWriter

```
import java.io.FileWriter;
import java.io.IOException;

public class EscribirTexto {
    public static void main(String[] args) {
        try (FileWriter fw = new FileWriter("archivo.txt")) {

            fw.write("Hola alumnos\n");
            fw.write("Segunda línea");

        } catch (IOException e) {
            System.out.println("Error al escribir");
        }
    }
}
```

4. Sobrescribir vs añadir

Igual que antes:

```
FileWriter fw = new FileWriter("archivo.txt", true);
```

✓ Añade contenido

✗ Sin `true` → sobrescribe

5. Problema: leer carácter a carácter es ineficiente

Solución: usar buffers

6. BufferedReader (lectura eficiente)

Permite leer **líneas completas**, que es lo más común.

Ejemplo clave

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class LeerLineas {
```

```

public static void main(String[] args) {
    try (BufferedReader br = new BufferedReader(new FileReader("archivo.txt"))) {

        String linea;

        while ((linea = br.readLine()) != null) {
            System.out.println(linea);
        }

    } catch (IOException e) {
        System.out.println("Error al leer");
    }
}

```

Explicación:

- `readLine()` → lee una línea completa
- Devuelve `null` al terminar

Esto es lo MÁS usado en Java real

7. Escribir eficientemente (BufferedWriter)

```

import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class EscribirLineas {
    public static void main(String[] args) {
        try (BufferedWriter bw = new BufferedWriter(new FileWriter("archivo.txt"))) {

            bw.write("Primera línea");
            bw.newLine();
            bw.write("Segunda línea");

        } catch (IOException e) {
            System.out.println("Error");
        }
    }
}

```

8. La clase File

No sirve para leer/escribir directamente.

Sirve para **gestionar archivos y carpetas**

Crear un objeto File

```
import java.io.File;
```

```
File archivo = new File("archivo.txt");
```

9. Información de un archivo

```
File archivo = new File("archivo.txt");
```

```
System.out.println("Existe: " + archivo.exists());  
System.out.println("Nombre: " + archivo.getName());  
System.out.println("Ruta absoluta: " + archivo.getAbsolutePath());  
System.out.println("Tamaño: " + archivo.length());
```

10. Crear y borrar archivos

```
File archivo = new File("nuevo.txt");
```

```
// Crear  
archivo.createNewFile();
```

```
// Borrar  
archivo.delete();
```

 `createNewFile()` lanza excepción → usar try/catch

11. Trabajar con directorios

```
File carpeta = new File("miCarpeta");
```

```
// Crear carpeta  
carpeta.mkdir();
```

```
// Listar contenido  
String[] archivos = carpeta.list();
```

```
for (String nombre : archivos) {  
    System.out.println(nombre);  
}
```

```
}
```

12. Rutas relativas vs absolutas

✓ Ruta relativa

```
File f = new File("archivo.txt");
```

Depende del directorio del proyecto

✓ Ruta absoluta

```
File f = new File("C:\\Users\\Gabriel\\archivo.txt");
```

13. Ejemplo completo (muy importante)

Leer un archivo y copiarlo línea a línea

```
import java.io.*;
```

```
public class CopiarTexto {
    public static void main(String[] args) {
        try (
            BufferedReader br = new BufferedReader(new FileReader("origen.txt"));
            BufferedWriter bw = new BufferedWriter(new FileWriter("destino.txt"))
        ) {

            String linea;

            while ((linea = br.readLine()) != null) {
                bw.write(linea);
                bw.newLine();
            }

            System.out.println("Copia realizada");

        } catch (IOException e) {
            System.out.println("Error");
        }
    }
}
```

Resumen

- `Reader` / `Writer` → trabajan con **texto**
- `FileReader` / `FileWriter` → básicos
- `BufferedReader` → leer líneas (MUY importante)
- `BufferedWriter` → escribir eficiente
- `File` → gestionar archivos/carpetas
- Mejor usar siempre buffers para texto

EJERCICIOS

1. Leer archivo línea a línea

Usa `BufferedReader`.

2. Contar líneas

Cuenta cuántas líneas tiene un archivo.

3. Contar palabras

Cuenta palabras separadas por espacios.

4. Contar caracteres

Incluyendo espacios.

5. Copiar archivo con `BufferedWriter`

Copiar `origen.txt` a `destino.txt`.

6. Convertir a mayúsculas

Leer archivo y guardarlo todo en mayúsculas.

7. Filtrar líneas

Copiar solo líneas que contengan la palabra "Java".

8. Añadir contenido

Pedir texto al usuario y añadirlo a un archivo.

9. Crear directorio y archivo

Crear carpeta `datos` y dentro un archivo.

10. Listar archivos de carpeta

Mostrar todos los archivos de un directorio.